



BidPilot Pakistan

AI-Powered Proposal & Bid Response Engine



A full-stack web application that reads any RFP or tender document, extracts every requirement using a 3-pass AI pipeline, matches them against a company's capability library, produces a section-by-section editable proposal draft, and calculates a defensible GO/NO-GO win probability — all in minutes.

PROJECT NAME	BUILD TYPE	DATE	TARGET MARKET
BidPilot Pakistan	Hackathon Project	June 2026	Pakistani Bidders

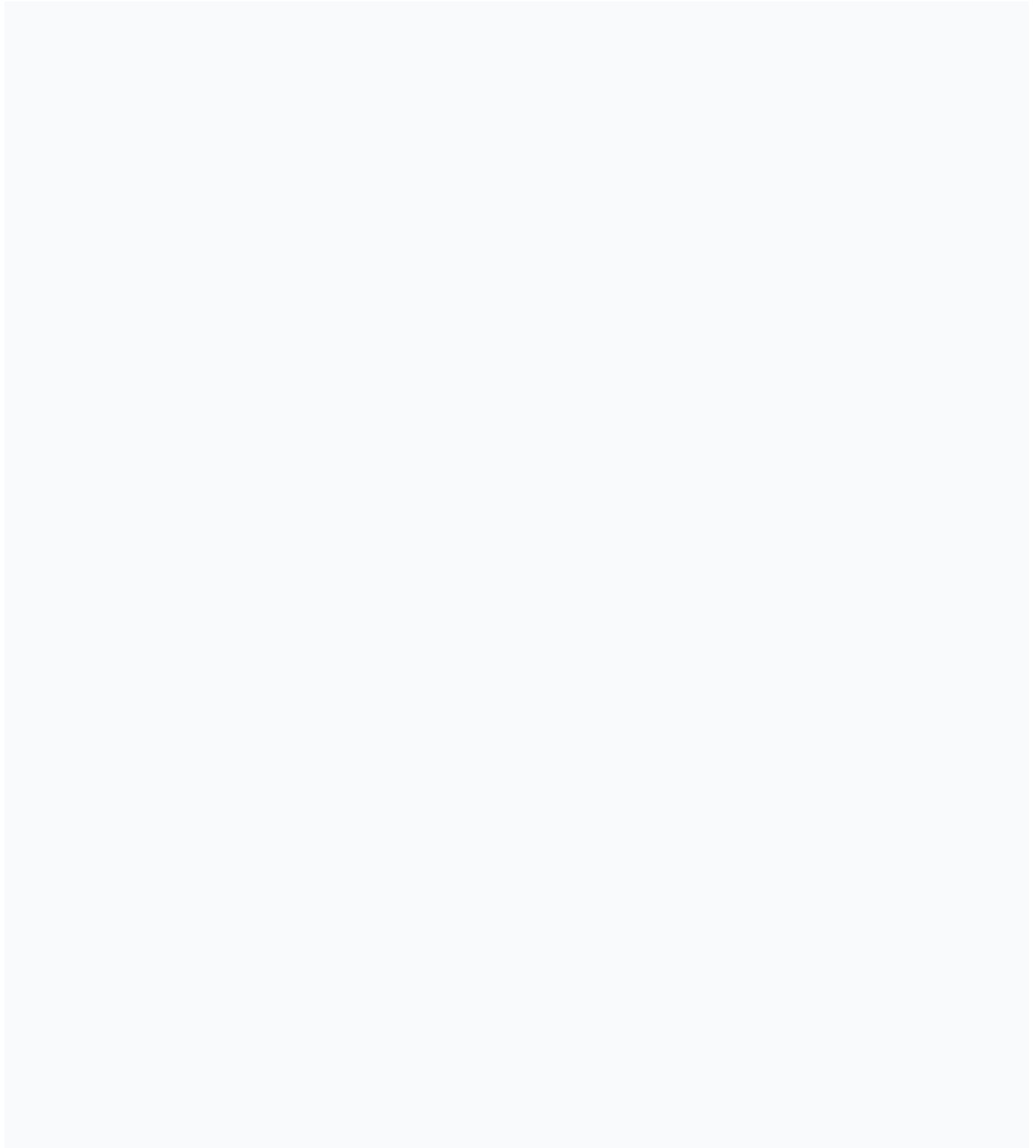


Table of Contents

1

Problem Statement & Solution Overview

2

Technology Stack

3

System Architecture

4

Authentication & User Management

5

Company Profile & Onboarding

6

RFP Workspace — Upload & Parse

7

3-Pass Deep AI Analysis Engine

8

Compliance Matrix

9

Proposal Studio — Section-by-Section Generation

10

Proposal Improvement Engine

11

Win Score & GO/NO-GO Calculator

12

Gap Analysis Module

13

Capability Library

14

Proposal Export (HTML Print-to-PDF)

15

AI Proposal Style Profiles

16

Anti-Hallucination Safety System

17

Full API Reference

18

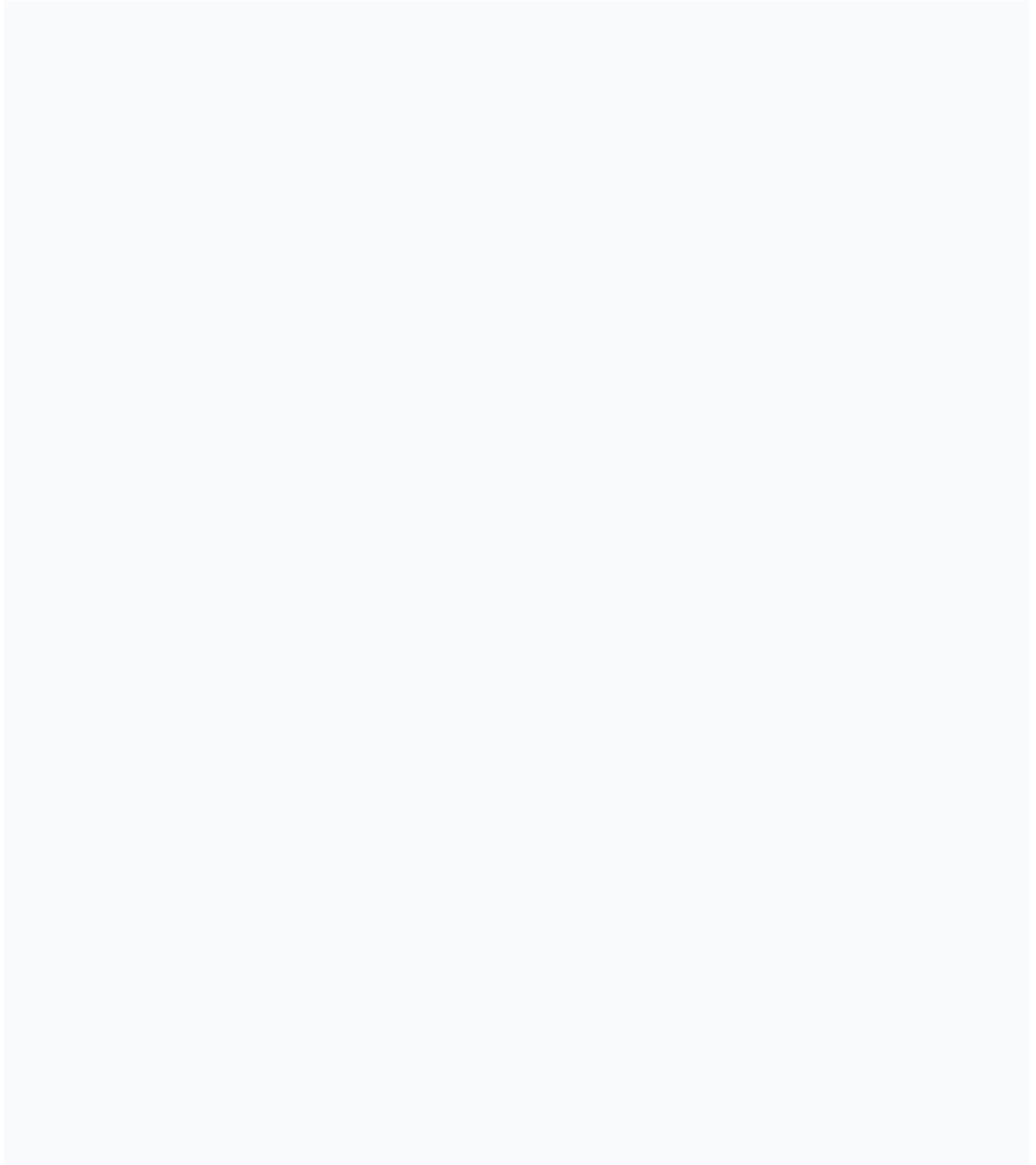
Data Flow & Storage

19

Bug Fixed During Development

20

Key Differentiators & Unique Features



1

Problem Statement & Solution Overview

Why BidPilot exists and what it does

The Problem

Businesses in Pakistan — software houses, engineering firms, consultancies, logistics companies — regularly bid on government and private tenders. The process is deeply painful:

- **RFP documents are dense** — government tenders can be 50–200 pages of legal, technical, and financial requirements buried in complex language.
- **Missing a mandatory clause = disqualification.** A single overlooked requirement means the entire bid is thrown out, regardless of how good the proposal is.
- **Proposals are rewritten from scratch every time** — even when 80% of the content is the same company profile, methodology, and team structure.
- **Generic AI tools hallucinate** — they invent certifications, client names, project values, and compliance claims that do not exist, creating legal and credibility risk.
- **No traceability** — it is impossible to know which part of the proposal addresses which RFP clause.

The Solution — BidPilot Pakistan

BidPilot is a full-stack AI web application that takes a tender document (PDF, DOCX, or pasted text) and produces a complete, traceable, compliance-mapped proposal draft — grounded entirely in the bidder's own evidence, with no fabricated claims.

**Extract**

3-pass AI analysis extracts every requirement, clause, deadline, evaluation criterion, and disqualification risk from the RFP.

**Match**

Each requirement is matched against the company's real capabilities, certifications, and past projects. Gaps are flagged explicitly.

**Generate**

Section-by-section proposal draft is generated in the correct Pakistani tender style for the company's sector, with evidence traceability and a hallucination risk score per section.

Target Users

- Pakistani **software houses** and IT firms bidding on PPRA, PITB, and government IT tenders
- **Engineering and construction firms** bidding on NHA, NESPAK, and infrastructure projects
- **Consulting firms** bidding on World Bank, ADB, and government advisory contracts
- **Logistics companies** bidding on supply-chain and transport tenders
- Any company that regularly responds to RFPs and wants to win more while spending less time writing

Core Value Proposition

3-Pass

AI ANALYSIS

100%TRACEABLE
CLAIMS**0**FABRICATED
FACTS**5**SECTOR
PROFILES

2

Technology Stack

Every library, framework, and service used

Frontend & Framework

Next.js 15 (App Router) — React full-stack framework. Handles both the UI and all backend API routes in a single project.

React 19 — UI component library with hooks.

TypeScript — Full type safety across frontend and backend.

Tailwind CSS v4 — Utility-first styling with a dark, professional design system.

shadcn/ui — Accessible, pre-built components (Cards, Buttons, Badges, Progress bars, Sheets, Toasts).

Lucide React — Icon library.

Authentication & Database

Firebase Authentication — Email/password and Google OAuth sign-in. Manages user sessions via HTTP-only cookies.

Firebase Admin SDK — Server-side session verification. Used in the protected route layout to redirect unauthenticated users.

Browser localStorage — All user data (company profile, workspaces, proposals) is stored locally per user UID. No cloud database required — keeps the architecture simple for the hackathon.

Document Parsing

pdf-parse (v1.1.1) — Extracts raw text from PDF files. Fixed a well-known Next.js compatibility bug (see Section 19).

mammoth (v1.12.0) — Extracts raw text from DOCX (Microsoft Word) files.

Built-in text handling — Plain `.txt` files are read directly as UTF-8.

AI & Language Model

Groq API — Ultra-fast LLM inference API. Uses the `llama-3.3-70b-versatile` model (70-billion parameter Llama 3.3). Groq's custom hardware (LPU) delivers responses 10x faster than OpenAI at a fraction of the cost.

Temperature is carefully tuned per endpoint:

- `0.05` for structured extraction (must be deterministic)
- `0.1` for analysis and planning
- `0.2-0.25` for proposal writing (slight creativity)

Validation & Utilities

Zod — Runtime schema validation on every API route. Ensures all incoming request bodies are correctly shaped before hitting the AI.

clsx / tailwind-merge — Utility for conditional class name merging.

Sonner — Toast notification library.

useSyncExternalStore (React) — Real-time sync of localStorage state across all open tabs.

Hosting & Deployment

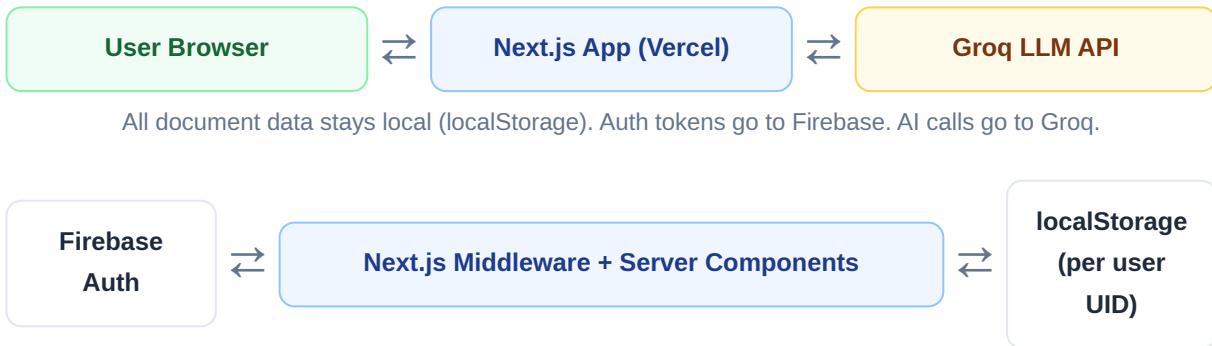
Built to deploy on **Vercel** (Next.js native platform). All API routes use `export const runtime = "nodejs"` to run as serverless Node.js functions. The `maxDuration` is set to 60–120 seconds on AI-heavy routes to handle long LLM calls.

3

System Architecture

How all components connect

High-Level Architecture



Directory Structure

```

src/
  app/
    (auth)/      ← Login, Signup, Forgot Password pages
    (app)/       ← Protected pages (requires Firebase session)
      dashboard/ ← Command center with global metrics
      company-settings/ ← 6-step company onboarding form
      company-profile/ ← Capability extraction & display
      capability-library/ ← Full capability cards view
      workspaces/ ← RFP workspace list
      workspaces/new/ ← Upload/paste RFP + deep analyze
      workspaces/[id]/ ← Per-tender workspace
        analysis/ ← Extracted requirements view
        compliance/ ← PASS/PARTIAL/FAIL matrix
        proposal/ ← Proposal Studio editor
        win-score/ ← GO/NO-GO probability score
        gaps/ ← Gap analysis dashboard
    api/         ← Backend API routes (serverless functions)
      rfp/parse/ ← Document text extraction
      rfp/analyze/ ← Quick RFP overview
      rfp/deep-analyze/ ← 3-pass deep extraction
  
```

```

proposal/plan/ ← AI section planning
proposal/generate/ ← Full proposal draft
proposal/generate-section/ ← Single section AI
proposal/improve-section/ ← Section rewrite AI
export/proposal/ ← Premium HTML export

components/
  auth/          ← Auth forms, Google button, provider
  proposal/      ← ProposalEditor, GeneratorPanel
  app/           ← AppShell (sidebar), PageHeader
  ui/            ← shadcn/ui base components

lib/
  ai/groq.ts     ← Groq API wrapper
  client-storage.ts ← localStorage + React hooks
  proposal-style.ts ← 5 sector style profiles
  premium-proposal-template.ts ← HTML export renderer
  constants.ts   ← Routes, GO/NO-GO bands, sectors

```

Request Flow

1

User visits the app

Next.js server-side layout checks Firebase session cookie. If no valid session → redirect to `/login`.

2

User creates a workspace (uploads RFP)

Browser sends PDF/DOCX/TXT to `POST /api/rfp/parse`. Server extracts raw text using pdf-parse or mammoth. Returns paginated document object.

3

Deep AI analysis runs

Browser sends RFP text to `POST /api/rfp/deep-analyze`. Server makes 3 sequential Groq calls (Pass 1 → overview, Pass 2 → requirements, Pass 3 → missed-item audit). Returns structured JSON.

4

Data saved to localStorage

Workspace with requirements, deep analysis, and parsed pages is saved to `localStorage` under a key namespaced to the user's Firebase UID. Reactive hooks re-render UI automatically.

5

Proposal generation

In the Proposal Studio, the user triggers section generation. Browser calls `POST /api/proposal/generate-section` per section. Returns HTML content with evidence metadata and hallucination risk score.

4

Authentication & User Management

Firebase-powered secure sessions

Sign-up Flow

Users register at `/signup` with email + password. Firebase creates the account. The app then calls `POST /api/auth/signup` to create an HTTP-only session cookie from the Firebase ID token. On success, the user is redirected to the dashboard.

Login Flow

Users sign in at `/login` with email + password, or via Google OAuth (one-click Google sign-in). Firebase returns an ID token which is exchanged for a server-side session cookie via `POST /api/auth/session`.

Password Reset

The `/forgot-password` page uses Firebase's built-in `sendPasswordResetEmail`. Users receive a reset link by email managed entirely by Firebase.

Session Management

HTTP-only cookie stores the Firebase session token. This cookie is checked server-side in the `(app)/layout.tsx` using Firebase Admin SDK's `verifySessionCookie`. Unauthenticated requests are redirected to `/login` before any page renders.

The `AuthProvider` client component makes the Firebase user object available to all child components via React Context.

Data Isolation

Every `localStorage` key is namespaced to the user's Firebase UID:

- `bidpilot:{uid}:company` — company profile
- `bidpilot:{uid}:workspaces` — all workspaces
- `bidpilot:{uid}:proposal:{workspaceId}` — proposal drafts

This ensures no data leaks between accounts on the same device.

Routes

Protected routes: `/dashboard` ,
`/company-settings` , `/company-profile` , `/capability-library` ,
`/workspaces` , `/settings` ,
`/government-rfps`

Auth routes: `/login` , `/signup` ,
`/forgot-password`

5

Company Profile & Onboarding

6-step structured company intelligence gathering

The company profile is the foundation of every proposal. BidPilot uses it to match requirements, populate proposal sections, calculate the win score, and assess document readiness. No two accounts share data.

The 6-Step Onboarding Form

Step 1 — Basic Info

- Company name & trading name
- Category (IT / Construction / Consulting / etc.)
- Sector, city, country
- Website URL, contact person, email, phone

Step 2 — Legal & Tax

- NTN (National Tax Number)
- STRN (Sales Tax Registration)
- SECP company number
- Active Taxpayer status, PEC category
- Upload confirmation flags for tax & registration docs

Step 3 — Services & Sectors

- Main services & secondary services
- Industries served, geographic coverage
- Client type (government / private / both)
- Typical project size, years in business, team size

Step 4 — Past Projects

- Multiple past projects with client name, type, sector
- Project value, year completed, duration
- Scope summary, evidence uploaded flag
- Performance certificate available flag

Step 5 — Certifications

- ISO certifications, other certifications
- Vendor registrations (PPRA, etc.)
- Financial docs & bank statements availability

Step 6 — Team & Assets

- Number of employees, key experts
- Software tools used
- Logo URL, cover page URL, letterhead URL

Capability Extraction

On the Company Profile page, the user can click **Analyze** to run `inferCapabilities()`, a client-side function that reads the company profile and generates a structured capability library:

- Detects sector keywords (construction, software, logistics, consulting) using regex
- Generates typed capability objects: `industry_experience`, `delivery_methodology`, `technical_skill`, `compliance_document`
- Each capability has a **confidence score** (0–1) and evidence text
- Capabilities are displayed as cards and fed into every proposal section generated

6

RFP Workspace — Upload & Parse

Getting the RFP text into the system accurately

Creating a Workspace

At `/workspaces/new`, users provide:

- **Workspace title** — e.g. "Peshawar Ring Road Technical Proposal"
- **Issuing authority** — e.g. "NHA", "PPRA", "Private Client"
- **Sector** — e.g. "Construction", "IT", "Logistics"
- **RFP document** — Upload a PDF, DOCX, or TXT file *or* paste raw text

Document Parser — `POST /api/rfp/parse`

This endpoint handles both file uploads (`multipart/form-data`) and direct text pastes (`text/plain`).

Input Type	Library Used	What Happens
<code>.pdf</code>	pdf-parse (internal lib)	Binary buffer converted to text. All pages extracted. Returns clean string.
<code>.docx</code>	mammoth	Word document XML unpacked, raw text extracted without formatting.
<code>.txt</code> / <code>text/*</code>	Node.js Buffer	UTF-8 decoded directly.
Plain text paste	Built-in	Request body read as text, no parsing needed.

What the Parser Returns

A `ParsedDocument` object containing:

- **fullText** — Complete cleaned text (newlines normalised, excess blank lines collapsed)
- **pages** — Array of `ParsedPage` objects, each ~3,000 characters, with extracted headings and table snippets
- **markdown** — Same as fullText (for display)
- **metadata** — File name, type, total pages, character count, estimated reading time

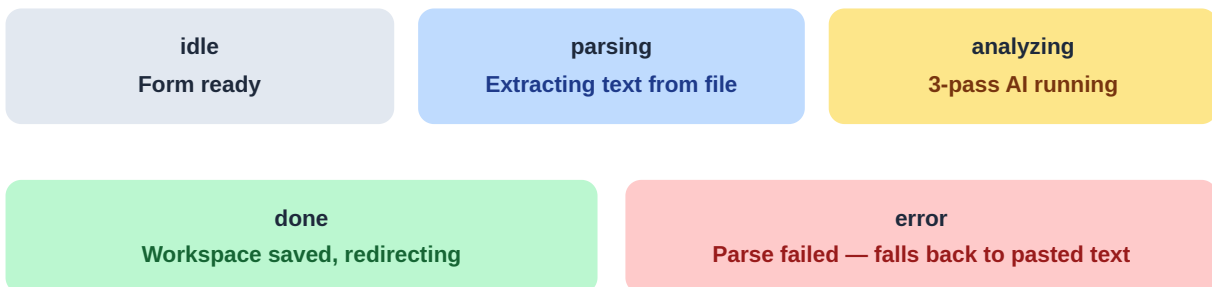
Heading & Table Extraction

Even before the AI sees the document, the parser runs local heuristics:

- **Headings** detected by: short all-caps lines, lines matching `#` markdown prefix, or lines ending with a colon
- **Tables** detected by: consecutive lines containing `|` or `\t` characters — extracted as table snippets

This gives the AI better context about document structure even before a full analysis pass.

Workspace States



Graceful fallback: If file parsing fails (e.g. corrupted PDF), the error is shown but the form continues with the manually pasted text. The workspace is not lost.

7

3-Pass Deep AI Analysis Engine

The most technically sophisticated feature — `POST /api/rfp/deep-analyze`

This is BidPilot's core intelligence. Three sequential AI calls process the same RFP text with increasing depth. Each pass has a different role. Combined, they produce a comprehensive requirement registry with traceability to source sections.

Pass 1 — Document Overview (temperature 0.05)

System prompt: "You are a Pakistani procurement document analyst. Extract structured tender overview data as strict JSON. Never invent missing details — use null or [] when unavailable."

What it extracts:

- Tender title
- Issuing organisation
- Submission deadline
- Pre-bid meeting date
- Bid security amount
- Budget / contract value
- Contract duration
- Scope of work (summary)
- Deliverables list
- Submission method
- Contact details
- Evaluation criteria + weights
- Required documents, certifications, experience, team roles
- Disqualification clauses

Pass 2 — Requirement Registry (temperature 0.05)

System prompt: "You are a Pakistani procurement requirements analyst. Extract every requirement from the tender text as a JSON array. Each requirement must be specific, traceable, and non-duplicated."

Each requirement object contains:

Field	Type	Purpose
id	string	REQ-001, REQ-002... for traceability
requirementText	string	Exact quote from the tender
normalizedRequirement	string	Clear restatement in plain language
category	enum	mandatory / technical / financial / eligibility / compliance / experience / team / document / other
mandatory	boolean	Is this a must-have or disqualification risk?
sourcePage	string null	Which page it came from
sourceSection	string null	Which section/clause
evidenceNeeded	string null	What document/proof is required
disqualificationRisk	boolean	Does failing this mean instant disqualification?
proposalSectionImpacted	string null	Which section of the proposal should address this

Pass 3 — Missed Requirement Audit (temperature 0.05)

System prompt: "You are a senior Pakistani procurement auditor performing a missed-requirement review. Identify anything missed: hidden eligibility clauses, submission conditions, disqualification risks, date/deadline items, evaluation sub-criteria, or required forms."

Specifically checks for:

- Hidden eligibility clauses buried in fine print
- Submission envelope/format requirements
- Bid validity period
- Performance bond / guarantee requirements
- Liquidated damages clauses
- Language or format requirements
- Required annexures or forms not in the main list
- Dates and deadlines not captured in Pass 1

Missed items are appended to the requirements array with `sourceSection: "Missed – audit pass"` for clear attribution.

Analysis Confidence Score

Automatically calculated based on how many requirements were extracted:

- **≥ 5 requirements extracted:** 82% confidence
- **1–4 requirements:** 60% confidence
- **0 requirements:** 20% confidence (document may be unreadable)

8

Compliance Matrix

Requirement-by-requirement PASS/PARTIAL/FAIL tracking

The compliance matrix at `/workspaces/[id]/compliance` is the most operationally critical view — it shows exactly which RFP requirements the company can meet, partially meet, or cannot meet, with evidence and suggested corrective actions.

Status Values

PASS

Company
evidence directly
matches this
requirement

PARTIAL

Some evidence
exists but gaps
remain — needs
strengthening

FAIL

No matching
evidence —
mandatory
failures =
disqualification
risk

UNKNOWN

Requirements
from AI deep-
analyze — not yet
matched against
profile

What the Matrix Shows Per Requirement

Column	Content
Requirement ID	REQ-001, REQ-002... sourced from deep analysis
Requirement text	Exact text from RFP
Status badge	PASS / PARTIAL / FAIL / UNKNOWN with colour coding
Matched evidence	What in the company profile supports this requirement
Suggested action	Specific next step to close the gap
Confidence	0–100% reliability score for the match

How Requirements Are Matched

When the deep AI analysis succeeds, all requirements start as **UNKNOWN** (the AI identified the requirement but hasn't yet matched it to company capabilities). The user must:

1. Review each requirement in the compliance tab
2. Update the company profile with relevant evidence
3. Use the Proposal Studio to generate sections that address each requirement directly

When the AI analysis fails or the document is pasted as text, the `extractRequirements()` function runs a local heuristic match using the company profile's category, sector, and services keywords.

9

Proposal Studio — Section-by-Section Generation

The heart of BidPilot — where the proposal is built

The Proposal Studio at `/workspaces/[id]/proposal` is a full document editor powered by a two-step AI pipeline: **Plan** then **Generate**.

Step 1 — AI Proposal Planning (`POST /api/proposal/plan`)

Before writing anything, the AI decides *what* to write. The planner receives:

- The RFP text
- RFP-specified required sections (from deep analysis)
- Evaluation criteria
- Mandatory requirements
- Sector information

And returns a `ProposalPlan` object with every section rated:

Priority	Meaning
must-have	RFP explicitly requires this section — omitting it is disqualifying
recommended	Strongly improves the proposal's score even if not explicitly required
optional	Nice to have but won't significantly impact the score
skip	Not relevant for this tender type (e.g. IT-specific sections in a construction bid)

The 16 default sections the planner chooses from:

- Cover Page
- Table of Contents
- Executive Summary
- Appreciation of the Project
- Team Structure and Key Personnel
- Quality Assurance
- Risk Management
- Compliance Matrix

- Understanding of Scope and Requirements
 - Approach and Methodology
 - Technical Work Plan
 - Implementation Timeline
- Financial Proposal
 - Company Profile and Relevant Experience
 - Certifications and Registrations
 - Annexures and Required Documents

Step 2 — Section Writing (`POST /api/proposal/generate-section`)

Each section is written individually by the AI. The API receives:

- **sectionTitle** & **sectionPurpose**
- **rfpText** — the relevant RFP excerpt
- **companyName** & **companyProfile**
- **relevantRequirements** — which REQ-IDs this section must address
- **capabilities** — extracted capability evidence
- **styleNotes** — sector-specific writing style guidelines

What Each Generated Section Contains

Field	Description
<code>html</code>	The actual section content as HTML, ready for display and export
<code>evidenceUsed</code>	List of evidence items that grounded this section's claims
<code>requirementsCovered</code>	Array of REQ-IDs this section addresses
<code>missingEvidence</code>	List of evidence items the AI needed but didn't have
<code>confidenceScore</code>	0.0–1.0 score for how well-grounded this section is
<code>hallucinationRisk</code>	LOW / MEDIUM / HIGH — how likely AI fabricated something
<code>improvementSuggestion</code>	Specific advice on what to add to strengthen this section

10

Proposal Improvement Engine

POST /api/proposal/improve-section — rewrite any section on demand

After a section is generated, users can click **Improve** to have the AI rewrite it based on the identified weaknesses. This is a targeted rewrite — not a full regeneration.

How It Works

1

User reviews a generated section

The Proposal Studio shows the HTML content, confidence score, hallucination risk, and improvement suggestion from the initial generation.

2

User clicks "Improve"

The browser sends the current HTML, known improvement suggestion, missing evidence list, and all relevant requirements to the improve-section API.

3

AI rewrites with precision

The improve API uses a specialised system prompt: *"You are BidPilot Pakistan, a senior proposal editor. Fix weak evidence gaps — but never invent facts. Where evidence is still missing, write [Evidence required: what is needed]."*

4

Updated section replaces the original

The new HTML, updated confidence score, and revised improvement suggestion are saved back to localStorage and displayed.

What the Improvement API Receives

Parameter	Purpose
sectionTitle	Which section is being improved
currentHtml	The existing (weak) content to rewrite
improvementSuggestion	Specific feedback from the initial generation
missingEvidence	List of evidence gaps to explicitly address

Parameter	Purpose
rfpText	Context from the original RFP
companyProfile	Updated company information
capabilities	Company capability evidence
relevantRequirements	REQ-IDs and text for compliance grounding

Safe fallback: If the JSON parse of the AI response fails, the original HTML is returned unchanged rather than showing broken content. The user sees "Try regenerating — the AI response was malformed."



11

Win Score & GO/NO-GO Calculator

Defensible bid probability from 6 weighted factors

The Win Score page at `/workspaces/[id]/win-score` calculates a 0–98% probability of winning the tender using a transparent, weighted formula. Every point is traceable to a specific data input.

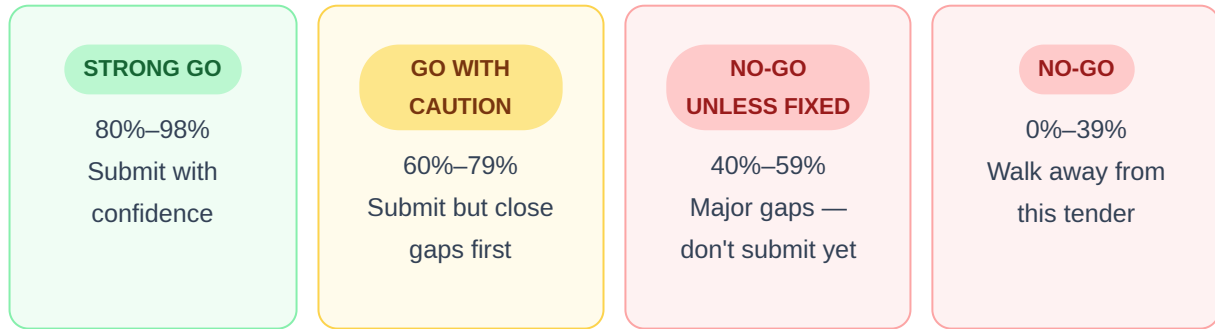
Scoring Formula (100 points total)

Component	Max Points	How Calculated
Mandatory compliance	35 pts	$(\text{Mandatory PASSes} / \text{total mandatory requirements}) \times 35$
Overall compliance	25 pts	$(\text{PASS requirements} / \text{all requirements}) \times 25$
Document readiness	15 pts	Points for NTN (+20), SECP (+15), tax docs (+15), reg docs (+15), financials (+15), bank statements (+10), performance certs (+10) — scaled to 15
Partial credit	10 pts	$(\text{PARTIAL requirements} / \text{all requirements}) \times 10$
Sector match	10 pts	10 pts if RFP sector matches company sector; 5 pts if partial match; 0 if no match
Certification fit	5 pts	5 pts if ISO, PEC, or other certifications are present; 0 otherwise

Disqualification Penalty

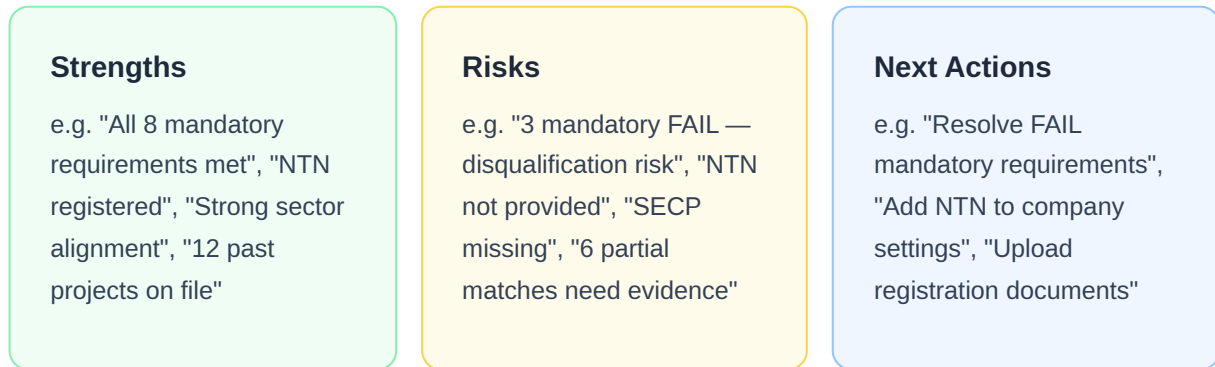
For each mandatory requirement that has a **FAIL** status, **8 points are deducted** from the total. This penalty is capped at 40 points. This directly models how procurers disqualify bids that miss mandatory clauses.

Decision Bands



Automatically Generated Insights

The Win Score page also generates three lists automatically:



12-
16

Gap Analysis, Capability Library, Export, Style Profiles & Anti-Hallucination

Supporting features that make BidPilot production-grade

12 — Gap Analysis Module

The Gap Analysis page at `/workspaces/[id]/gaps` surfaces all non-PASS requirements in a prioritised view, helping the team focus on the most critical work before submission. High-risk gaps (mandatory FAILs) are shown with red alerts at the top.

13 — Capability Library

At `/capability-library`, all extracted capabilities from the company profile are displayed as cards. Each card shows:

- **Title** — e.g. "Civil engineering and infrastructure delivery"
- **Type** — industry_experience / delivery_methodology / technical_skill / compliance_document
- **Evidence** — the specific text grounding the claim
- **Confidence** — 0–100% reliability score
- **Tags** — searchable keywords

These capabilities are automatically injected into every proposal section generation request.

14 — Proposal Export (Premium HTML Print-to-PDF)

The `GET /api/export/proposal` route (and the print button in the Proposal Studio) renders a premium standalone HTML document using `renderPremiumProposalHtml()`. This HTML is:

- A4-formatted, print-safe, with proper CSS `@media print` rules
- Contains a professional cover page, company branding section, all proposal sections
- Includes a compliance matrix table at the end

- Status badges (PASS/PARTIAL/FAIL/UNKNOWN) styled with colour coding
- Confidence scores and evidence status per section
- Can be opened in any browser and printed directly to PDF — no server-side PDF generation needed

15 — AI Proposal Style Profiles

BidPilot automatically detects the sector from the company category and RFP text and applies a matching writing style profile. Five profiles exist:

Sector	Detected By	Key Sections Added
Construction	Keywords: construction, highway, road, bridge, engineering, civil, motorway, dfbot	Appreciation of project, design standards, surveys, traffic management, QA/HSE plan
IT / Software	Keywords: software, IT, technology, SaaS, cloud, ERP, app, AI, cyber, data	Solution architecture, security & compliance, support SLA, technical approach
Consulting	Keywords: consulting, advisory, feasibility, study, capacity building	Diagnostic approach, workstreams, governance, deliverable register
Logistics	Keywords: logistics, supply chain, transport, fleet, warehouse	Operational plan, fleet/resources, routing, safety, SLA commitments
Services	Default for all other sectors	Scope response, methodology, quality plan, staffing

16 — Anti-Hallucination Safety System

Every AI prompt in BidPilot contains explicit anti-hallucination guardrails. These are enforced at the system prompt level on every single AI call:

- **"Never invent certifications, client names, staff CVs, project values, or compliance evidence."**
- **"If evidence is missing, write exactly: [Evidence required: describe what is missing]"**
- **"Avoid: cutting-edge, seamless, robust, leverage, comprehensive solution, innovative platform, world-class"** (generic filler language that sounds like hallucination)
- **"Use: specific dates, named deliverables, traceable methodology, clear compliance language"**
- The `hallucinationRisk` field (LOW/MEDIUM/HIGH) is returned on every section and shown in the UI
- Temperature 0.05–0.25 is deliberately low to prevent creative fabrication

17

Full API Reference

Every backend route, its input, and its output

RFP Routes

Route	Method	Input	Output	AI Calls
/api/rfp/parse	POST	multipart file (PDF/DOCX/TXT) or text/plain body	ParsedDocument with pages, fullText, metadata	0 — local only
/api/rfp/analyze	POST	rfpText (min 50 chars), sector	analysis JSON with summary, deadlines, budget, requirements	1 (Groq)
/api/rfp/deep-analyze	POST	rfpText (min 30 chars), sector	DeepAnalysisResult: full overview + requirements registry + missed audit	3 (Groq) sequential

Proposal Routes

Route	Method	Input	Output	AI Calls
/api/proposal/plan	POST	rfpText, requiredSections[], evaluationCriteria[], mandatoryRequirements[], sector	ProposalPlan: sections with priority, word count, rationale	1 (Groq)
/api/proposal/generate	POST	companyName, companyCategory, rfpText, companyEvidence	Full Markdown proposal draft + category + style label	1 (Groq)
/api/proposal/generate-section	POST	sectionTitle, sectionPurpose, rfpText, companyName, companyProfile,	GenerateSectionResponse: html, evidenceUsed[], requirementsCovered[], missingEvidence[],	1 (Groq)

Route	Method	Input	Output	AI Calls
		relevantRequirements[], capabilities, styleNotes	confidenceScore, hallucinationRisk, improvementSuggestion	
/api/proposal/improve-section	POST	sectionTitle, currentHtml, improvementSuggestion, missingEvidence[], rfpText, companyProfile, capabilities, relevantRequirements[]	Same as generate-section response	1 (Groq)

Auth & Export Routes

Route	Method	Purpose
/api/auth/signup	POST	Creates Firebase session cookie from ID token after registration
/api/auth/session	POST	Creates/refreshes Firebase session cookie for login
/api/export/proposal	GET	Returns standalone print-ready HTML proposal with demo data
/api/examples/proposal-pdf	GET	Example proposal render for testing

Groq API Wrapper (src/lib/ai/groq.ts)

All AI calls go through a single shared function `callGroq(messages, options)` . It:

- Reads `GROQ_API_KEY` and `GROQ_MODEL` environment variables
- Sends a `POST` to `https://api.groq.com/openai/v1/chat/completions`
- Returns the raw text content from `choices[0].message.content`
- Throws a descriptive error if the API responds with a non-200 status
- Marked with `import "server-only"` — TypeScript will error if imported client-side

18

Data Flow & Storage

How data moves and where it lives

Storage Architecture

BidPilot uses **browser localStorage** as its primary data store. This is an intentional choice for the hackathon — no cloud database is needed, the app works offline, and data is completely private to the device. Each key is namespaced by Firebase UID to prevent cross-account leakage.

Data Types Stored

Type	localStorage Key	Contents
StoredCompany	<code>bidpilot: {uid}:company</code>	Full company profile including all 6 onboarding steps, past projects, certifications, generated capabilities
StoredWorkspace[]	<code>bidpilot: {uid}:workspaces</code>	Array of all workspaces — each contains: title, issuer, sector, rfpText, requirements[], deepAnalysis, parsedPages
StoredProposalDraft	<code>bidpilot: {uid}:proposal: {workspaceId}</code>	All proposal sections with HTML, confidence, evidence status, hallucination risk, approved flag

Real-Time State Sync

BidPilot uses React's `useSyncExternalStore` with a custom event (`bidpilot-storage`) to make localStorage reactive. When any function saves data to localStorage:

1. It dispatches a `bidpilot-storage` custom event
2. All subscribed `useSyncExternalStore` hooks re-run their snapshot function

3. Any component depending on that data re-renders automatically

This works across browser tabs too via the native `storage` event.

JSON Cache Layer

A `Map<string, {raw: string; value: unknown}>` cache sits in front of `localStorage.getItem()`. It compares the raw string to avoid re-parsing JSON on every render. This prevents performance issues when hooks run frequently.

Dashboard Metrics (Computed, Not Stored)

The Dashboard page computes all metrics in a `useMemo` — they are never stored separately:

- **Total workspaces** — `workspaces.length`
- **Avg compliance score** — average PASS rate across all workspaces
- **Avg win probability** — weighted score: $\text{compliance} \times 0.7 + \text{partial credit} \times 0.2 + \text{base } 10$
- **High-risk tenders** — workspaces where any mandatory requirement is FAIL or UNKNOWN
- **Open gaps** — total non-PASS requirements across all workspaces

19

Critical Bug Fixed During Development

pdf-parse crashing all PDF uploads with HTTP 500

The Bug

Every PDF upload was returning a **500 Internal Server Error**. The error message was a Node.js ENOENT (file not found) error for a file path like `./test/data/05-versions-space.pdf`.

Root Cause

The `pdf-parse` npm package (v1.1.1) has a known compatibility bug with Next.js. The package's entry point (`index.js`) contains this code at the module level:

```
// node_modules/pdf-parse/index.js
const Fs = require('fs');
const Pdf = require('./lib/pdf-parse.js');
module.exports = Pdf;

let isDebugMode = !module.parent; // ← THE BUG

if (isDebugMode) {
  let PDF_FILE = './test/data/05-versions-space.pdf';
  let dataBuffer = Fs.readFileSync(PDF_FILE); // ← CRASHES HERE
}
```

In **Node.js CommonJS**, `module.parent` is set to the requiring module. But in **Next.js's bundler** (webpack/turbopack), `module.parent` is `undefined`. So `!module.parent` evaluates to `true`, and the debug block runs — trying to read a test PDF that doesn't exist in production.

The Fix

Import the internal implementation directly, bypassing the buggy index entry point entirely:

```
// ❌ BEFORE – imports the buggy index.js
const mod = await import("pdf-parse");

// ✅ AFTER – imports the pure parser function directly
const mod = await import("pdf-parse/lib/pdf-parse.js" as any);
const pdfParse = (mod as any).default ?? mod;
const result = await pdfParse(buffer);
```

The file `node_modules/pdf-parse/lib/pdf-parse.js` is the actual parsing function with **no side effects** — it just exports the function and nothing else. This fix is permanent and does not require any package version changes.

Impact

This bug was silently blocking **all PDF uploads**. After the fix, PDF parsing works correctly for all file sizes and types. DOCX and TXT parsing were unaffected as they do not use pdf-parse.

20

Key Differentiators & Unique Features

What makes BidPilot different from generic AI tools

1. 3-Pass Analysis (No Generic AI Tool Does This)

Most AI tools send the document once and hope for the best. BidPilot makes 3 sequential AI calls with different expert roles: analyst, requirements extractor, and auditor. The third pass specifically hunts for what the second pass missed.

2. Hallucination Risk per Section

Every generated section has a LOW/MEDIUM/HIGH hallucination risk label with an explanation. This is unique — other proposal tools don't tell you when to be suspicious of the AI's output.

3. Evidence Traceability

Every claim in the proposal is linked back to either a company profile field or a REQ-ID from the RFP. If evidence is missing, the AI writes `[Evidence required: ...]` rather than fabricating content.

4. Sector-Specific Writing Styles

Construction bids require completely different language, sections, and tone from IT proposals. BidPilot detects the sector automatically and applies a pre-defined style profile tailored to Pakistani procurement standards.

5. Disqualification Risk Detection

The system explicitly flags requirements where failure means instant disqualification — before the user invests days writing a proposal that will be thrown out. The Win Score applies an 8-point penalty per failed mandatory requirement.

6. Pakistani Procurement Context

Every AI prompt is calibrated for Pakistani procurement: NTN, STRN, SECP, PEC categories, PPRA compliance, NHA projects — the system understands local context that generic AI tools ignore.

7. Multi-Workspace Bid Pipeline

Unlike one-shot tools, BidPilot maintains a full pipeline of active tenders. The dashboard shows aggregate metrics across all workspaces so bid managers can prioritise effort across multiple simultaneous tenders.

8. Proposal Improvement Loop

The improve-section API creates a human-in-the-loop feedback cycle. Users review AI output, identify weak points, click Improve, and the AI rewrites specifically to address the identified weaknesses — not a full regeneration.

Summary of What Was Built

In summary: BidPilot Pakistan is a full-stack Next.js 15 application with Firebase authentication, Groq LLM integration, document parsing (PDF/DOCX/TXT), a 3-pass deep AI analysis engine, a section-by-section proposal editor with improvement loop, a 6-factor win probability calculator, sector-specific writing style profiles, anti-hallucination guardrails, and a premium print-to-PDF export system — all built and debugged for the hackathon.

8

API ROUTES

11

APP PAGES

5

AI PERSONAS

3

DOC FORMATS